CS3103 Assignment 4: LiveRecall

Aw Ming Xuan, Kosolpattanadurong Thitiwat, Lee Zhao Rui, Wong Ji Kean

1 Architecture and Protocol

Our project splits into two parts: the solution server and the client. The solution server has two main components: a `CaptionStore`, which holds the captions received from the caption server, and a list of `Client` objects, each holding a client's websocket connection and its send queue.

Upon receiving the websocket initialized message, our server use `msg_type` to detect whether it is a request from the caption server or client (`SOURCE_HELLO` or `CLIENT_HELLO`). Then, our server has two separate handlers (`handle_source` and `handle_client_new`) to handle the websocket. The following sections will explain how each handler is implemented internally.

1.1 Server: storing captions

We utilize a `CaptionStore` object (`server/store.py`) holding the server's view of the transcript within `keep_seconds` seconds range. To achieve this, we maintain a deque of segment numbers `order` to hold the transcript within `keep_seconds` seconds and a hashmap from segment number to the `Entry` which holds the information of the segment, including `source_ts` (source timestamp) and the text.

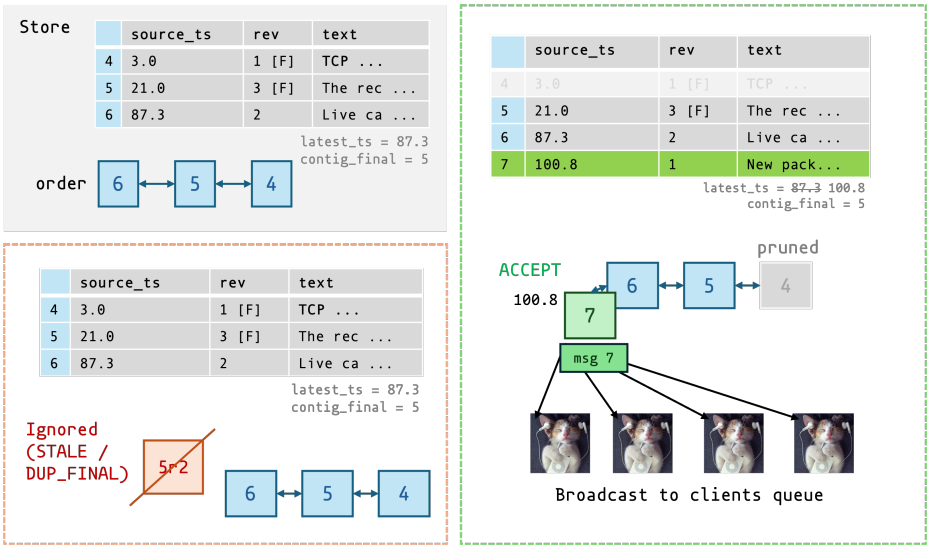


Figure 1: `CaptionStore` implementation

When our backend receives a `CAPTION_UPDATE` message from the caption server, we parse and validate it into an `Entry` (malformed messages are counted and dropped), then call `store.apply(entry)`, which returns either `STALE`,`DUP_FINAL`,`TOO_OLD`, or `ACCEPTED` based on our **revision semantics**.

- A revision is only accepted if it is strictly newer than the one we hold (`new.revision > cur.revision`); older or repeated revisions are `STALE` and dropped, so a delayed packet can never roll a caption back.
- Any update for a segment that is *already-final*[1] returns `DUP_FINAL`.
- Any update for a segment that does not exist on a deque is considered `TOO_OLD`.
- Otherwise, we treat it as `ACCEPTED`.

For all `ACCEPTED` requests, we update the store and broadcast the captions to clients. It is very important to keep the store up-to-date (and atomic) because a client that reconnects (and sends a `HISTORY` request) will ask for the segments

[1] Checking *already-final* is tricky: since pruning removes segments from memory, we cannot simply look a segment up to know if it was finalized. We therefore keep `contig_final`, the highest N such that segments $1..N$ are all final. As such, a late update for a segment at or below `contig_final` is rejected as `DUP_FINAL`.

it lacks (via `get_history()`). While updating the store, we also prune the outdated segments (segment that is older than `now - keep_seconds` seconds) to bound the size of our deque.

Recovery semantics

When a client disconnects, it reconnects immediately on `close` (except when the server closes it with code `1008`) and sends a `CLIENT_HELLO` carrying its resume state: `last_segment`, `last_revision` and `history_seconds` (60 by default). `get_history()` then walks the deque from newest to oldest and returns the finalized entries the client lacks in temporal order, skipping anything already held (older than `last_segment`, or not newer than `last_revision`), provisional, or older than `history_seconds`.

The `HISTORY` message is queued before the client joins the broadcast set, so it always precedes live updates; any overlap is deduplicated by the client's revision rule. A late joiner uses the same path with `last_segment = 0`.

1.2 Server: Sending to clients

1.3 Client (`app.js`)

(TODO: discuss on backpressure)

1.4 Overload policy

1.5 Optimization attempts

2 Results

3 Discussion

4 Conclusion

A Raw data